MOHAMED BIN ZAYED UNIVERSITY OF ARTIFICIAL INTELLIGENCE

Mohamed bin Zayed University of Artificial Intelligence

SWM Engineering

Date
18/2/2026

6.1

Software Testing

Why Testing?

- **My code is perfect. There are no bugs.**
- **Really?**

Stage	Typical Bugs per 1000 LOC (KLOC)
During initial coding	15 – 50 defects
After testing (release quality)	0.5 – 5 defects
Safety-critical systems	< 0.1 defects

LOC: Lines of Code

Remember: More code more bugs!

How many of all bugs do we find? https://software-lab.org/publications/ase2018_static_bug_detectors_study.pdf

But it is just a Bug!

Year	System / Event	Type of Bug	Impact	Estimated Loss
1985–87	Therac-25 Radiation Machine	Race condition	Radiation overdoses (6+ deaths)	**Human lives**
1996	Ariane 5 Flight 501	Integer overflow	Rocket exploded 37 sec after launch	~$370M
1999	Mars Climate Orbiter	Unit conversion error	Spacecraft destroyed	~$125M
2012	Knight Capital Trading	Deployment / logic bug	Erroneous trades in 45 min	~$440M
2014	Heartbleed (OpenSSL)	Missing bounds check	Global data exposure	Millions affected
2014	Apple “goto fail” Bug	Duplicate statement	SSL security bypass	Security risk worldwide
2018	Windows 10 Update	File handling bug	User data deletion	Reputational + financial loss
2021	Facebook Global Outage	Configuration error	6-hour worldwide outage	~$100M+

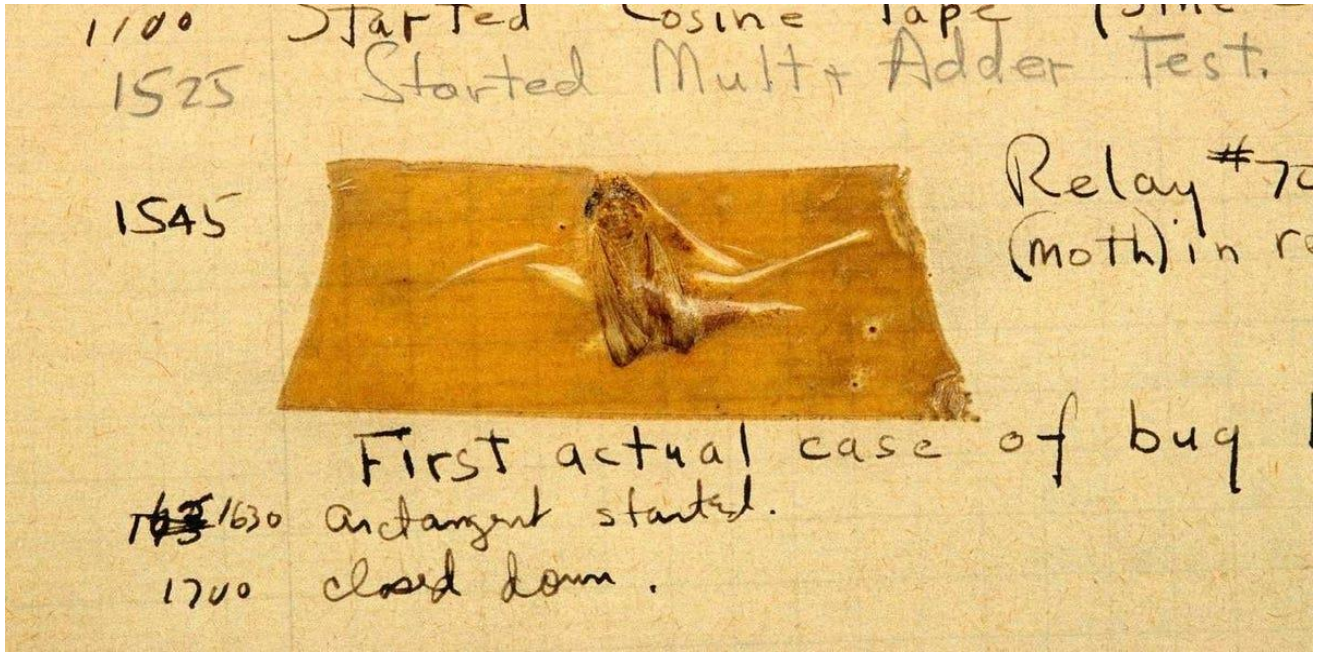
But it is just a Bug!

Year	System / Event	Type of Software Bug	Impact	Estimated Fatalities
1985–1987	Therac-25 Radiation Machine	Race condition / lack of synchronization	Massive radiation overdoses	6+ deaths
1996	Ariane 5 Flight 501	Integer overflow (data conversion error)	Rocket explosion shortly after launch	No direct deaths (unmanned)
1991	Patriot Missile Failure (Gulf War)	Floating-point rounding error	Missile failed to intercept incoming Scud	28 soldiers killed
2007	Airbus A330 Qantas Flight 72	Faulty sensor data + software response	Sudden aircraft dive	0 deaths, 100+ injured
2018–2019	Boeing 737 MAX (MCAS system)	Faulty sensor input handling + control logic	Two crashes (Lion Air & Ethiopian Airlines)	346 deaths

But it is just a Bug!

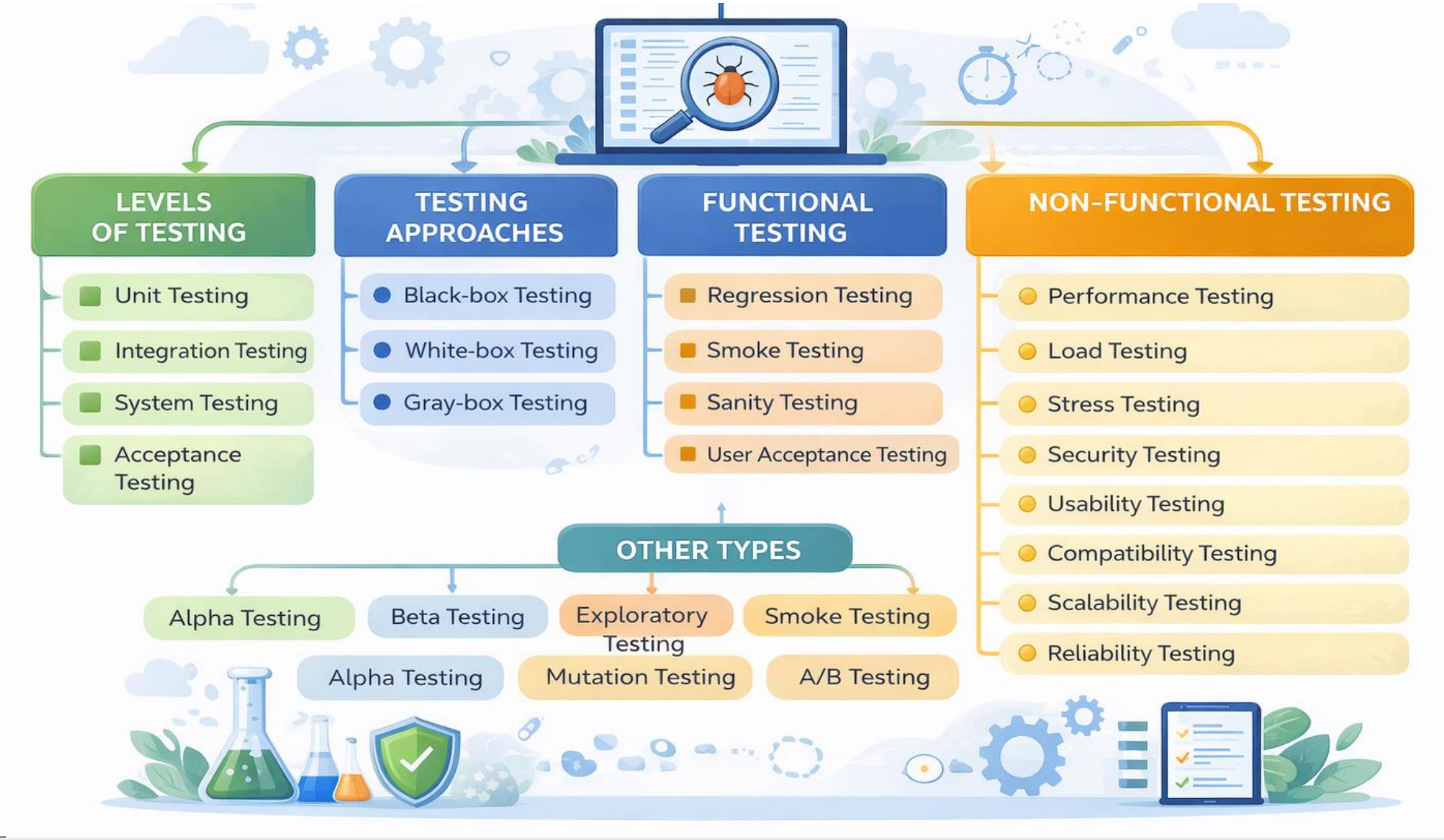
Software may contain bugs.

Software testing is the systematic process of **evaluating a software system** to verify that it **meets the specified requirements** and to **identify defects** before release.



Harvard Mark II Moth (the first Bug), September 9, 1947

Types of Testing



At a Glance

Testing Approaches

- **Black Box**: Tests what the system does.
- **White Box**: Tests how the system works.
- **Gray Box**: Tests both behavior and partial internal logic.

Functional Testing

- **Regression Testing** – Re-testing existing features to ensure new changes did not break anything.
- **Smoke Testing** – Basic check to see if the main functions of the application work after a new build.
- **Sanity Testing** – Quick test to verify a specific bug fix or small change works correctly.
- **User Acceptance Testing (UAT)** – Testing performed by end users to confirm the system meets business requirements.

Non-Functional Testing

- **Performance Testing** – Measures speed and responsiveness of the system.
- **Load Testing** – Checks system behavior under expected user load.
- **Stress Testing** – Tests system under extreme or peak conditions.
- **Security Testing** – Identifies vulnerabilities and protects against threats.
- **Usability Testing** – Evaluates user-friendliness and user experience.
- **Compatibility Testing** – Ensures system works across devices, browsers, and OS.
- **Scalability Testing** – Tests system ability to handle growth in users/data.
- **Reliability Testing** – Verifies system stability over time.
- **Availability Testing** – Ensures system uptime and fault tolerance.
- **Recovery Testing** – Checks how well the system recovers from crashes or failures.

Purpose

Testing is not only about discovering bugs, but also about checking if the software meets the requirements (functional or non-functional).

Unit Testing

Unit testing is the process of testing individual components or functions of a program in isolation to verify that they work correctly.

Key Characteristics

- Tests the **smallest testable parts** (functions, methods, classes)
- Usually written and executed by **developers**
- Performed **early** in development
- Often automated
- Focuses on **internal logic**

Purpose

- Detect bugs early
- Ensure each unit behaves as expected
- Simplify debugging
- Support refactoring safely

Unit Testing Example

```
def add(a, b):
    return a + b


def test_add():
    assert add(2, 3) == 5


test_add()   # Manual call
```

Unit Testing

As code becomes more complex, we need tools for a more robust and helpful testing environment.

Important notes:

- Test edge cases
- Test error conditions
- Keep tests small and focused
- Avoid testing implementation details

Unit Testing: `unittest`

`unittest` is Python's **built-in unit testing framework**, inspired by JUnit (a unit testing framework for Java). It is part of the Python standard library — no installation required.

It allows you to:

- Write automated tests
- Organize tests into test cases
- Verify expected behavior using assertions
- Run test suites automatically

Unit Testing: `unittest`

`unittest` is Python's **built-in unit testing framework**, inspired by JUnit (a unit testing framework for Java). It is part of the Python standard library — no installation required.

It allows you to:

- Write automated tests
- Organize tests into test cases
- Verify expected behavior using assertions
- Run test suites automatically

A unit test consists of:

- A **test class** (inherits from `unittest.TestCase`)
- One or more **test methods**
- **Assertions** to verify expected behavior

Unit Testing: unittest

```python
# math_utils.py
def multiply(a, b):
 return a * b
```

```python
# test_math_utils.py
import unittest
from math_utils import multiply

class TestMathUtils(unittest.TestCase):

    def test_multiply_positive(self):
        self.assertEqual(multiply(3, 4), 12)

    def test_multiply_zero(self):
        self.assertEqual(multiply(5, 0), 0)

if __name__ == "__main__":
    unittest.main()
```

Running the tests

```
python test_math_utils.py

python -m unittest discover
```

Each method that begins with `test_` is executed automatically.

Unit Testing: `unittest`

Assertions verify correctness

```
assertEqual(a, b)
assertNotEqual(a, b)
assertAlmostEqual(first, second, delta=0.001)
assertTrue(x)
assertFalse(x)
assertIsNone(x)
assertIn(a, b)
assertRaises(Exception, function)
…
```

Unit Testing: unittest

Testing exceptions

```
# calculator.py
def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero is not allowed")
    return a / b
```

```
# test_calculator.py
import unittest
from calculator import divide

class TestCalculator(unittest.TestCase):

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            divide(10, 0)

if __name__ == "__main__":
    unittest.main()
```

```
def test_exception_message(self):
    with self.assertRaises(ValueError) as
context:
        divide(10, 0)

    self.assertEqual(str(context.exception),
"Division by zero is not allowed")
```

- `context.exception` stores the raised exception.
- we can inspect it.

Unit Testing: unittest

Setup and TearDown

Optional methods for preparing and cleaning up before and after each test

```
# bank.py
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
```

```
# test_bank.py
import unittest
from bank import BankAccount

class TestBankAccount(unittest.TestCase):

    def setUp(self):
        print("Setting up test environment")
        self.account = BankAccount(100)

    def tearDown(self):
        print("Cleaning up test environment")
        self.account = None

    def test_deposit(self):
        self.account.deposit(50)
        self.assertEqual(self.account.balance, 150)

    def test_withdraw(self):
        self.account.withdraw(40)
        self.assertEqual(self.account.balance, 60)
...
```

Unit Testing: `unittest`

Method	Runs	Use For
`setUp()`	Before each test	Fresh objects, test isolation
`tearDown()`	After each test	Cleanup
`setUpClass()`	Once before all tests	Expensive setup (DB connection)
`tearDownClass()`	Once after all tests	Closing shared resources

Unit Testing: `pytest`

`pytest` is a popular **third-party** **testing framework for Python** used to write and run automated tests. It is designed to be **simple** for small projects while remaining **powerful** and **scalable** for large systems.

It is widely used because it is:

- Simple
- Powerful
- Flexible
- Less verbose than `unittest`

```
>>> pip install pytest

>>> pytest
```

Unit Testing: pytest (example)

```
my_project/
│
├── calculator.py
└── test_calculator.py
```

```python
# calculator.py (code to test)
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by
zero")
    return a / b
```

```python
# test_calculator.py
import pytest
from calculator import add, divide


def test_add():
    assert add(2, 3) == 5


def test_divide():
    assert divide(10, 2) == 5


def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)
```

```
pytest
3 passed in 0.01s
```

Unit Testing: pytest

Parameterized Testing

@pytest.mark.parametrize is a powerful pytest **decorator** that enables data-driven testing by running a single test function multiple times with different input values.

```
@pytest.mark.parametrize("a,b,result", [
    (1, 1, 2),
    (2, 5, 7),
    (0, 0, 0)
])

def test_add_multiple(a, b, result):
    assert add(a, b) == result
```

Integration Testing

Unit Testing vs Integration Testing

- **Unit test:** tests one function/class **in isolation** (often with **mocks**).
- **Integration test:** tests real **interaction** between modules (often fewer mocks).

A **mock** is a fake object that simulates the behavior of a real dependency.

Instead of calling the real:
- Database
- API
- Email service
- File system
- Payment gateway

We replace it with a **controlled fake version**.

Integration Testing: example

Suppose we have the following components:

- `database.py`: stores user data
- `service.py` : business logic
- `app.py` : user registration

We want to test that:
When we **register** a user, the **service** correctly stores the data in the **database**.

```
project/
│
├── database.py
├── service.py
├── app.py
└── test_integration.py
```

Integration Testing: example

```python
# database.py
class UserDatabase:
    def __init__(self):
        self.users = {}

    def add_user(self, username, email):
        self.users[username] = email

    def get_user(self, username):
        return self.users.get(username)
```

```python
# service.py
from database import UserDatabase

class UserService:
    def __init__(self, db: UserDatabase):
        self.db = db

    def register_user(self, username, email):
        if not username or not email:
            raise ValueError("Invalid input")
        self.db.add_user(username, email)
```

Integration Testing: example

```python
# test_integration.py
import pytest
from database import UserDatabase
from service import UserService

def test_register_user_integration():
    db = UserDatabase()
    service = UserService(db)

    service.register_user("alice", "alice@example.com")

    # Verify interaction between service and database
    assert db.get_user("alice") == "alice@example.com"
```

Integration Testing: example

```python
# app.py
from database import UserDatabase
from service import UserService

def main():
    db = UserDatabase()
    service = UserService(db)

    username = input("Enter username: ")
    email = input("Enter email: ")

    service.register_user(username, email)
    print("User registered successfully!")

if __name__ == "__main__":
    main()
```

```
pytest

Or

pytest test_integration.py
```

Mohamed bin Zayed
University of
Artificial Intelligence

Thank you